OPERATING SYSTEM - PROJECT 1 REPORT

XV6 AND SYSTEM CALL IMPLEMENTATION

23125002 - 23125052

1. wc

1.1 Objective

The objective of this task is to extend the existing wc command in xv6 to support Linux-style command-line options. The modified wc command supports the following options:

- -l: count number of lines
- -w: count number of words
- -c: count number of characters

The command works with multiple files and standard input, while preserving the original behavior when no options are specified.

1.2 Design and implementation

Option parsing

Command-line options are parsed in the main() function by scanning arguments that start with the - character. Each valid option enables a corresponding global flag.

If no option is specified, the program defaults to printing all three counts (lines, words, and characters), which matches the original behavior of wc.

Invalid options are detected and reported immediately.

Counting logic

The core counting logic is implemented in a separate wc() function. Input is read using the read() system call in fixed-size buffers.

For each character read:

- The character count is incremented unconditionally.
- The line count is incremented when a newline character ('\n') is encountered.

- The word count is determined using a state variable that detects transitions from whitespace characters to non-whitespace characters.

This approach ensures correct word counting even when multiple spaces, tabs, or newline characters appear consecutively.

1.3 Input handling

The modified wc command supports both file input and standard input.

- If one or more filenames are provided, each file is opened, processed, and closed sequentially.
- If no filename is provided, the program reads from standard input, allowing wc to be used with pipes.

1.4 Output format

The output format follows the behavior of the Linux wc command:

- Only the requested counts are printed, in the order: lines, words, characters.
- When input is read from a file, the filename is printed at the end of the output line.
- Each result is printed on a separate line.

1.5 Test cases

Test case 1: Default behavior

```
$ wc README
```

Output

```
48 334 2425 README
```

Test case 2: Single option

```
$ wc -l README
```

Output

```
48 README
```

Test case 3: Multiple options

```
$ wc -cw README
```

Output

```
334 2425 README
```

Test case 4: Multiple files

```
wc -l README domains.txt
```

Output

```
48 README
4 domains.txt
```

Test case 5: Standard input

```
cat README | wc -w
```

Output

```
334
```

2. cat

2.1 Objective

The objective of this task is to extend the existing `cat` command in xv6 to support the Linux-style `-n` command-line option.

By default, the `cat` command outputs the contents of the input files or standard input unchanged. With the `-n` option enabled, each output line is prepended with a line number formatted according to the Linux `cat -n` specification.

The modified `cat` command supports:

- Multiple input files
- Standard input (stdin)
- Line numbering with fixed-width formatting
- Original behavior when the `-n` option is not specified

2.2 Design and implementation

Option parsing

Command-line arguments are parsed in the `main()` function.
If the first argument is `-n`, line numbering mode is enabled and subsequent arguments are treated as filenames.

As specified in the assignment, it is assumed that `-n`, if present, always appears as the first argument.

If no filenames are provided, the program reads from standard input.

Line-based input handling

The original xv6 `cat` implementation reads and writes raw byte buffers. To support line numbering, input must be processed one line at a time.

A custom `readline()` function is used to:

- Read characters one by one using the `read()` system call
- Stop reading when a newline character (`'\n'`) is encountered
- Properly terminate the line with a null character
- Handle end-of-file and error conditions correctly

This approach ensures accurate line detection for both files and standard input.

Line number formatting

Line numbering is implemented according to the Linux `cat -n` behavior:

- The line number column is exactly 8 characters wide
- 6 characters are reserved for the line number
- Followed by 2 spaces before the line content

A helper function `print_lineno()` computes the number of digits in the current line number and prints the required leading spaces to maintain proper alignment.

Line numbers are maintained using a static counter so that numbering continues sequentially across multiple files or standard input streams.

2.3 Input handling

The modified `cat` command supports both file input and standard input:

- If one or more filenames are provided, each file is opened, processed, and closed sequentially.
- If no filenames are provided, the program reads from standard input.
- Line numbering (when enabled) works consistently across multiple files and stdin.

2.4 Output format

The output format follows the behavior of the Linux `cat` command:

- Without `-n`, the output is identical to the original xv6 `cat`
- With `-n`, each line is prefixed with a right-aligned line number
- Line numbers are printed continuously across files
- Output lines are printed exactly as they appear in the input

2.5 Test cases

Test Case 1: Default behavior

```
$ cat domains.txt
```

Output

```
[google.com, 142.251.46.174]
[usfca.edu, 23.185.0.2]
[mit.edu, 104.90.21.210]
[openai.com, 13.107.238.57]
```

Test Case 2: Line numbering with `-n`

```
$ cat -n domains.txt
```

Output

```
     1  [google.com, 142.251.46.174]
     2  [usfca.edu, 23.185.0.2]
     3  [mit.edu, 104.90.21.210]
     4  [openai.com, 13.107.238.57]
```

Test Case 3: Multiple files

```
$ cat -n domains.txt README
```

Output

```
     1  [google.com, 142.251.46.174]
     2  [usfca.edu, 23.185.0.2]
     3  [mit.edu, 104.90.21.210]
```

```
     4  [openai.com, 13.107.238.57]
     5  xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's
Unix
     6  Version 6 (v6).  xv6 loosely follows the structure and style of v6,
     7  but is implemented for a modern RISC-V multiprocessor using ANSI C.
     8
     9  ACKNOWLEDGMENTS
    10
    11  xv6 is inspired by John Lions's Commentary on UNIX 6th Edition
(Peer
    12  to Peer Communications; ISBN: 1-57398-013-7; 1st edition (June 14,
    13  2000)).  See also https://pdos.csail.mit.edu/6.1810/, which
provides
    14  pointers to on-line resources for v6.
    15
    16  The following people have made contributions: Russ Cox (context
switching,
    17  locking), Cliff Frey (MP), Xiao Yu (MP), Nickolai Zeldovich, and
Austin
    18  Clements.
    19
    20  We are also grateful for the bug reports and patches contributed by
    21  Abhinavpatel00, Takahiro Aoyagi, Marcelo Arroyo, Hirbod Behnam,
Silas
    22  Boyd-Wickizer, Anton Burtsev, carlclone, Ian Chen, clivezeng, Dan
    23  Cross, Cody Cutler, Mike CAT, Tej Chajed, Asami Doi,Wenyang Duan,
    24  echtwerner, eyalz800, Nelson Elhage, Saar Ettinger, Alice Ferrazzi,
    25  Nathaniel Filardo, flespark, Peter Froehlich, Yakir Goaron, Shivam
    26  Handa, Matt Harvey, Bryan Henry, jaichenhengjie, Jim Huang, Matúš
    27  Jókay, John Jolly, Alexander Kapshuk, Anders Kaseorg, kehao95,
    28  Wolfgang Keller, Jungwoo Kim, Jonathan Kimmitt, Eddie Kohler, Vadim
    29  Kolontsov, Austin Liew, l0stman, Pavan Maddamsetti, Imbar
Marinescu,
    30  Yandong Mao, Matan Shabtay, Hitoshi Mitake, Carmi Merimovich,
    31  mes900903, Mark Morrissey, mtasm, Joel Nider, Hayato Ohhashi,
    32  OptimisticSide, papparapa, phosphagos, Harry Porter, Greg Price,
Zheng
    33  qhuo, Quancheng, RayAndrew, Jude Rich, segfault, Ayan Shafqat,
Eldar
    34  Sehayek, Yongming Shen, Fumiya Shigemitsu, snoire, Taojie, Cam
Tenny,
    35  tyfkda, Warren Toomey, Stephen Tu, Alissa Tung, Rafael Ubal,
unicornx,
```

```
    36  Amane Uehara, Pablo Ventura, Luc Videau, Xi Wang, WaheedHafez,
Keiichi
    37  Watanabe, Lucas Wolf, Nicolas Wolovick, wxdao, Grant Wu, x653, Andy
    38  Zhang, Jindong Zhang, Icenowy Zheng, ZhUyU1997, and Zou Chang Wei.
    39
    40  ERROR REPORTS
    41
    42  Please send errors and suggestions to Frans Kaashoek and Robert
Morris
    43  (kaashoek,rtm@mit.edu).  The main purpose of xv6 is as a teaching
    44  operating system for MIT's 6.1810, so we are more interested in
    45  simplifications and clarifications than new features.
    46
    47  BUILDING AND RUNNING XV6
    48
    49  You will need a RISC-V "newlib" tool chain from
    50  https://github.com/riscv/riscv-gnu-toolchain, and qemu compiled for
    51  riscv64-softmmu.  Once they are installed, and in your shell
    52  search path, you can run "make qemu".
```

Test Case 4: Standard input

```
$ cat README | cat -n
```

Output

```
     1  xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's
Unix
     2  Version 6 (v6).  xv6 loosely follows the structure and style of v6,
     3  but is implemented for a modern RISC-V multiprocessor using ANSI C.
     4
     5  ACKNOWLEDGMENTS
     6
     7  xv6 is inspired by John Lions's Commentary on UNIX 6th Edition
(Peer
     8  to Peer Communications; ISBN: 1-57398-013-7; 1st edition (June 14,
     9  2000)).  See also https://pdos.csail.mit.edu/6.1810/, which
provides
    10  pointers to on-line resources for v6.
    11
    12  The following people have made contributions: Russ Cox (context
switching,
```

locking), Cliff Frey (MP), Xiao Yu (MP), Nickolai Zeldovich, and Austin
Clements.

We are also grateful for the bug reports and patches contributed by
Abhinavpatel00, Takahiro Aoyagi, Marcelo Arroyo, Hirbod Behnam, Silas
Boyd-Wickizer, Anton Burtsev, carlclone, Ian Chen, clivezeng, Dan
Cross, Cody Cutler, Mike CAT, Tej Chajed, Asami Doi,Wenyang Duan,
echtwerner, eyalz800, Nelson Elhage, Saar Ettinger, Alice Ferrazzi,
Nathaniel Filardo, flespark, Peter Froehlich, Yakir Goaron, Shivam
Handa, Matt Harvey, Bryan Henry, jaichenhengjie, Jim Huang, Matúš
Jókay, John Jolly, Alexander Kapshuk, Anders Kaseorg, kehao95,
Wolfgang Keller, Jungwoo Kim, Jonathan Kimmitt, Eddie Kohler, Vadim
Kolontsov, Austin Liew, l0stman, Pavan Maddamsetti, Imbar Marinescu,
Yandong Mao, Matan Shabtay, Hitoshi Mitake, Carmi Merimovich,
mes900903, Mark Morrissey, mtasm, Joel Nider, Hayato Ohhashi,
OptimisticSide, papparapa, phosphagos, Harry Porter, Greg Price, Zheng
qhuo, Quancheng, RayAndrew, Jude Rich, segfault, Ayan Shafqat, Eldar
Sehayek, Yongming Shen, Fumiya Shigemitsu, snoire, Taojie, Cam Tenny,
tyfkda, Warren Toomey, Stephen Tu, Alissa Tung, Rafael Ubal, unicornx,
Amane Uehara, Pablo Ventura, Luc Videau, Xi Wang, WaheedHafez, Keiichi
Watanabe, Lucas Wolf, Nicolas Wolovick, wxdao, Grant Wu, x653, Andy
Zhang, Jindong Zhang, Icenowy Zheng, ZhUyU1997, and Zou Chang Wei.

ERROR REPORTS

Please send errors and suggestions to Frans Kaashoek and Robert Morris
(kaashoek,rtm@mit.edu). The main purpose of xv6 is as a teaching
operating system for MIT's 6.1810, so we are more interested in
simplifications and clarifications than new features.

BUILDING AND RUNNING XV6

You will need a RISC-V "newlib" tool chain from
https://github.com/riscv/riscv-gnu-toolchain, and qemu compiled for

```
riscv64-softmmu.  Once they are installed, and in your shell
search path, you can run "make qemu".
```

3. System call getprocs and ps

3.1 Objective

The objective of this task is to implement a new user-level command ps in xv6 that displays information about currently active processes, similar to the Linux ps command. To support this feature, a new system call getprocs() is introduced, allowing process information to be safely transferred from kernel space to user space.

3.2 Design overview

The design follows a kernel-managed data collection model combined with user-level presentation.

- The kernel is responsible for accessing the process table and collecting process metadata.
- The user program ps requests this information via a system call and formats the output.

This separation ensures safety, modularity, and consistency with xv6 design principles.

3.3 Shared data structure

A shared structure struct procinfo is defined to represent process information exchanged between kernel and user space. This structure includes the process ID, state, memory size, and name. The definition is placed in a shared header file so both kernel and user programs can interpret the data consistently.

3.4 System call architecture

The ps command relies on a newly added system call getprocs(). At the user level, getprocs() is declared as a regular C function that accepts:

- A pointer to an array of struct procinfo
- The maximum number of entries to retrieve

At the kernel level, the function receives:

- A raw virtual address (uint64) pointing to user memory
- The maximum number of processes to copy

This difference reflects the strict separation between user space and kernel space memory models in xv6.

3.5 Execution flow

The execution flow of the ps command is summarized as follows:

1. The user executes the ps command.
2. The user program allocates a buffer for process information.
3. The getprocs() system call is invoked.
4. Control transitions to kernel space via the syscall mechanism.
5. The kernel gathers process data from the process table.
6. Process information is copied safely to user memory.
7. Control returns to user space.
8. The ps program formats and prints the results.

3.6 Kernel implementation

3.6.1 System call handler

This function sys_getprocs() in kernel/sysproc.c acts as the system call entry point for getprocs(). Its responsibilities include:

- Retrieving system call arguments from user space
- Extracting the user buffer address and maximum entry count
- Delegating the actual work to the kernel-level getprocs() function

This layer separates syscall argument handling from process table logic.

3.6.2 Process table traversal

This function getprocs() in kernel/proc.c performs the core kernel-side logic of the ps command. Its main responsibilities are:

- Iterating through the global process table
- Skipping unused process entries
- Extracting relevant fields (PID, state, memory size, name)
- Copying each process record to user space using copyout()

By copying data one process at a time and validating memory access, the kernel ensures that user space cannot directly access or corrupt kernel memory.

3.7 User program implementation

The user program user/ps.c is responsible for presentation that includes the helper function statestr() converts internal process state values into human-readable strings.

The main() function performs the following tasks:

- Allocates an array of struct procinfo
- Calls getprocs() to retrieve process information
- Prints a formatted table containing PID, state, memory size, and name

All process information is obtained exclusively through the system call interface.

3.8 Output format

The output format closely follows the behavior of the Linux ps command:

- One process per line
- Fixed columns for PID, state, memory size, and name
- A header row describing each column

This consistent format improves readability and usability.

3.9 Summary

This task demonstrates how xv6 can be extended with a new system call and user-level utility.
It highlights key operating system concepts such as:

- User and kernel space separation
- Safe memory copying using copyout()
- System call design and dispatch
- Modular responsibility between kernel logic and user presentation

The ps command provides a clear and practical example of controlled kernel data exposure in xv6.

4. System call trace

4.1. Objective

The objective of this task is to extend the xv6 operating system kernel by implementing a new system call named trace. This system call provides a system call tracing mechanism, which is useful for debugging and understanding how user programs interact with the kernel.

The trace system call takes one integer argument called a mask. Each bit in the mask corresponds to a specific system call number. If the bit at position *n* is set, then the system call with number *n* will be traced. When a traced system call is about to return, the kernel prints:

- The process ID (PID)
- The name of the system call
- The return value of the system call

The tracing feature only affects the process that calls trace and any child processes created using fork(). Other processes in the system remain unaffected.

4.2. Adding trace to the build system

To make the trace program available in user space, it was added to the UPROGS list in the Makefile. This ensures that the trace user program is compiled and included in the xv6 file system image.

4.3. Declaring the new system call interface

In order to allow user programs to invoke the new system call, the following changes were made:

- A function prototype int trace(int); was added to user/user.h
- An entry entry("trace"); was added to user/usys.pl to generate the user-level system call stub
- A new system call number SYS_trace was added to kernel/syscall.h

These steps ensure that user programs can invoke the trace system call using the RISC-V ecall instruction.

4.4. Extending the process structure

A new field tracemask was added to the struct proc in kernel/proc.h. This field stores the system call tracing mask for each process. Storing the mask inside the process structure allows each process to maintain its own tracing configuration. This design also makes it possible for child processes to inherit the tracing behavior from their parent.

4.5. Implementing the sys_trace system call

The kernel-side implementation of the new system call was added to kernel/sysproc.c:

```
uint64 sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->tracemask = mask;
  return 0;
}
```

This function retrieves the mask argument from user space using argint(), then stores it in the tracemask field of the current process. From this point onward, the process and its future child processes will trace the system calls specified by the mask.

4.6. Registering the system call

In kernel/syscall.c, the system call was registered by:

1. Declaring the function prototype:

```
extern uint64 sys_trace(void);
```

2. Adding the system call to the system call dispatch table:

```
[SYS_trace] = sys_trace,
```

This allows the kernel to correctly dispatch the trace system call when it is invoked from user space.

4.7. Inheriting the trace mask in fork()

To ensure that child processes inherit the tracing behavior, the fork() function in kernel/proc.c was modified. After copying the parent process state to the child, the following line was added:

```
np->tracemask = p->tracemask;
```

This guarantees that any process created using fork() will continue tracing the same system calls as its parent.

4.8. System call name table

A system call name table was created in kernel/syscall.c to map system call numbers to human-readable names:

```
static char *syscall_names[] = {
  [SYS_fork]     = "fork",
  [SYS_exit]     = "exit",
  [SYS_wait]     = "wait",
  [SYS_pipe]     = "pipe",
  [SYS_read]     = "read",
  [SYS_kill]     = "kill",
  [SYS_exec]     = "exec",
  [SYS_fstat]    = "fstat",
  [SYS_chdir]    = "chdir",
  [SYS_dup]      = "dup",
  [SYS_getpid]   = "getpid",
  [SYS_sbrk]     = "sbrk",
  [SYS_pause]    = "pause",
  [SYS_uptime]   = "uptime",
```

```
  [SYS_open]     = "open",
  [SYS_write]    = "write",
  [SYS_mknod]    = "mknod",
  [SYS_unlink]   = "unlink",
  [SYS_link]     = "link",
  [SYS_mkdir]    = "mkdir",
  [SYS_close]    = "close",
  [SYS_getprocs] = "getprocs",
  [SYS_trace]    = "trace"
};
```

This table is used to print meaningful system call names during tracing.

4.9. Modifying the syscall() function

The syscall() function in kernel/syscall.c was modified to print tracing information when a system call is traced. After executing the system call and obtaining the return value, the kernel checks whether the corresponding bit is set in the process's tracemask.

For certain system calls such as read, write, and open, the parameters are also printed. For all other system calls, only the name and return value are displayed.

Example output format:

```
PID: syscall_name(arguments) -> return_value
```

This implementation ensures that tracing output is generated only for selected system calls.

4.10. User-level trace program

A user-level program trace.c was implemented to enable tracing for another program.

1. Parses the tracing mask from the command line
2. Calls the trace system call
3. Executes the target command using exec()

This allows tracing to be enabled without modifying the traced program itself.

4.11. Result

After completing the implementation, the trace command works as expected. For example:

```
$ trace 32 grep hello README
```

```
3: syscall read(1023, 0x0000000000002010, 1023) -> 1023
3: syscall read(965, 0x000000000000204a, 965) -> 965
3: syscall read(437, 0x000000000000204c, 963) -> 437
3: syscall read(0, 0x0000000000002010, 1023) -> 0
```

```
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open("'"'&'J'N'R'", 0) -> 3
4: syscall read(1023, 0x0000000000002010, 1023) -> 1023
4: syscall read(965, 0x000000000000204a, 965) -> 965
4: syscall read(437, 0x000000000000204c, 963) -> 437
4: syscall read(0, 0x0000000000002010, 1023) -> 0
4: syscall close -> 0
```